FAST National University of Computer and Emerging Sciences — Lahore

MuseFlow: AI-Powered Multi Genre Band Arrangement

Project Report - Artificial Intelligence

Abdul-Rehman Ahmad (24L-0555)| Raeyyan Habib (24L-0690)| Adam Hammad (24L-0678)

Academic Year 2025–2026 · Semester 4

I. Problem Definition & Domain Research

The Problem

A lot of independent musicians and hobbyists can come up with a melody — maybe hum once, play it on a keyboard, or sketch it as a MIDI — but they don't have the music theory knowledge or budget to build a full arrangement around it. Professional arrangements can cost hundreds of dollars per session, and tools like GarageBand still require you to know what you're doing harmonically.

MuseFlow solves this. You give it any melody and it automatically builds a full multi-instrument arrangement in Classical, Lofi, or Rock style, no music theory needed from the user. It also tackles the 'blank canvas' problem: the paralysis that comes from staring at an empty project. MuseFlow gives you a starting point almost always instantly.

Where the Data Came From

We used two datasets:

- Lakh MIDI Dataset (LMD): Around 45,000 MIDI files. We auto-classified them into Rock and Lofi using filename keywords and MIDI program numbers (e.g. distorted guitar = Rock, electric piano = Lofi). We used up to 4,000 files per genre.
- music21 Bach Chorales: A set of J.S. Bach four-part harmonisations from the music21 MIT corpus. These formed our Classical subset and were ideal because they already had clean melody/band track separation.

Dataset Details

Total files: up to 12,000 (4,000 per genre). Tokenisation: REMI scheme via MidiTok. Each sample is a 128-token melody prompt paired with a 384-token band target (512 tokens total). Vocabulary size: ~10,003 tokens, including 3 genre control tokens — 10000 for Classical, 10001 for Lofi, 10002 for Rock.

Potential Biases

- Single-track Lofi files: Many Lofi MIDI files had only one track, which broke our Seq2Seq pairing. We wrote a rescue.py script to synthetically split them.
- Genre label noise: LMD has no official genre tags, so our keyword-based classification could mislabel edge cases.
- Western bias: Both datasets are overwhelmingly Western tonal music. The model won't perform well on Arabic, Indian, or other non-Western musical systems.
- Melody extraction heuristic: We picked the melody track as the one with the highest average MIDI pitch. This works most of the time but isn't perfect.

II. Methodology & Design Choices

Preprocessing

We couldn't feed raw MIDI files into the network directly, so we built a pipeline to convert them into integer token sequences:

- Each file is loaded with symusic.Score. Files with fewer than 2 tracks are skipped.
- The track with the highest average pitch is treated as the melody prompt; everything else becomes the band target.
- Both are tokenised using MidiTok's REMI scheme with chord annotations, program numbers, velocity bins, and tempo events.
- The genre control token is prepended to the band sequence — this is how the model knows what genre to generate.
- Prompt sequences are padded/truncated to 128 tokens; targets to 384. Padding uses integer 0, which CrossEntropyLoss ignores via ignore_index=0.
- At inference, uploaded audio or MIDI is cleaned through clean_melody_score(), it picks the busiest track, removes very short notes, and snaps onsets to an 8th-note grid.

We used integer 0 for padding instead of a separate [PAD] token because REMI ID 0 is a null event that never appears in real music. Setting ignore_index = 0 gives us a masked loss for free, without adding to the vocabulary.

Why a Transformer?

We evaluated three model families:

- RNNs (LSTM/GRU): Rejected. They can't model long-range dependencies well. At 512 tokens, vanishing gradients make it hard to attend to the start of a melody phrase.
- CNNs (WaveNet-style): Rejected. These work best on raw audio waveforms, not discrete token sequences, and need very deep stacks to cover a 512-token window.
- Decoder-only Transformer: Selected. Self-attention handles all pairwise relationships regardless of distance. The causal mask gives us autoregressive generation, and genre tokens slot in naturally as control signals.

Final architecture: 4 layers, 8 attention heads, embedding dimension of 256, 512-token context window. Optimizer: AdamW (lr = 3e-4, weight decay = 0.01). LR schedule: Cosine Annealing. Total parameters: ~10 million. At inference, we use temperature T=0.85, top-k=50, and top-p=0.95 sampling to keep output varied without going off the rails.

III. Failure Analysis

Failure 1 : 6-Layer Model Overfit Immediately

We initially assumed more layers would produce better arrangements. We trained a 6-layer version under the same conditions.

What happened: By epoch 5 training loss was at ~0.72, but validation perplexity started going up. By epoch 12 the model was reproducing near-exact fragments from training files. Generated output sounded fine in isolation but was basically the same regardless of the input melody, genre conditioning had completely collapsed.

Why: With only ~3,000 usable training pairs, a 15M-parameter model had more than enough capacity to memorise the data instead of learning the actual melody-to-band mapping.

Fix: Dropped to 4 layers (~10M parameters), added L2 regularisation via weight_decay=0.01, and expanded the dataset to 2,000 files per genre.

Failure 2 : Single-Sequence Training Produced Genre Soup

Our first approach concatenated all tracks into one long sequence and appended the genre tag at the end, training the model to predict the next token throughout.

What happened: There was almost no genre differentiation. Asking for Rock often gave something nearly identical to Classical.

Why: In a left-to-right causal model, by the time the genre token appears at the end of a 512-token sequence, the model has already made most of its decisions. The tag came too late to matter.

Fix: Redesigned the whole pipeline as proper Seq2Seq. The genre token is now prepended at position 129 — the very start of the band sequence. Every generated band token is conditioned on the genre from the beginning.

IV. Results & Hyperparameter Tuning

Configuration Comparison

Baseline v1 (4 layers, d_model=128, 25 epochs, 500 files/genre) — Loss ~1.85. Thin embeddings, poor harmony.

Overfit v2 (6 layers, d_model=256, 25 epochs, 500 files/genre) — Loss ~0.72 but memorised the training set.

Stable v3 (4 layers, d_model=256, 25 epochs, 1,000 files/genre) — Loss ~1.42. Good genre separation.

Final v4 (4 layers, d_model=256, 50 epochs, 4,000 files/genre) — Loss ~1.18. Best generalisation and musical cohesion.

Training Dynamics (Final Model)

Epochs 1–10: Loss dropped from ~2.1 to ~1.6. The model picked up basic note event structure quickly.

Epochs 11–30: Loss 1.6 to ~1.3. Genre conditioning started solidifying.

Epochs 31–50: Loss 1.3 to 1.18. Fine-grained harmonic learning as the LR decayed.

We used mixed FP16 precision to halve VRAM usage and gradient clipping at max_norm=1.0 to prevent exploding gradients early on.

Evaluation Metrics

We combined token-level and music-domain metrics since the output is symbolic MIDI:

- Token-level accuracy: 61.3% (v3) → 67.8% (v4 final)
- Genre Conditioning Precision (dominant MIDI program matches expected genre): 74% (v3) → 89% (v4)
- Pitch class histogram correlation vs. training set: 0.71 → 0.84
- Note density (target ~4 notes/bar): 3.1 → 3.9
- Rhythmic grid adherence (post-quantisation): 98%

Post-Processing

The raw Transformer goes through several cleanup steps before being saved as a MIDI file:

- Track consolidation: merges fragmented same-program tracks into single instrument tracks.
- Vertical alignment: shifts all band tracks so they start at tick 0 alongside the melody.
- Duration clipping: trims all accompaniment tracks to exactly the melody's length.
- 16th-note quantisation: snaps onsets and durations to a 16th-note grid to remove rhythmic drift.
- Velocity normalisation: remaps velocities to 60 + (v × 0.4) to smooth out dynamic spikes.
- Humaniser: adds ±5% random duration jitter after quantisation so the output doesn't sound robotic.

V. Ethical Reflection & Limitations

Where It Would Fail

- Non-Western music: The model is trained entirely on Western tonal data. Arabic maqam, Indian ragas, or pentatonic inputs will be forced into Western harmonic structures.
- Atonal inputs: Anything not in 12-tone equal temperament gets snapped to the nearest tempered pitch, destroying intentional dissonance.

- Very short melodies: Looping tiles short inputs, but if there's not enough harmonic movement, the accompaniment gets repetitive fast.
- Rubato/free tempo: Basic Pitch transcribes with fixed tempo assumptions. Significant timing variation in a hummed recording will misalign with the model's grid-based training.
- Copyright: If someone hums a copyrighted melody, the generated arrangement could technically constitute a derivative work.

Ethical Implications

Training data copyright: LMD was scraped from publicly indexed MIDI files. They're widely used in research, but many are transcriptions of copyrighted songs. Training on them without explicit licensing sits in a grey area under current AI copyright law.

Musician displacement: MuseFlow can produce in seconds what a session arranger would spend hours on. At scale this devalues entry-level arranging and freelance session work — this is already happening on commercial stock music platforms with other AI tools.

Cultural misrepresentation: Our genre labels are broad and Western-centric. Calling something 'Classical' carries associations with European art music. Users from other backgrounds might assume the output reflects their own tradition.

Homogenisation: Democratising music production is genuinely useful, but if AI arrangements flood content platforms they'll all share the same statistical fingerprint. That could quietly reduce musical diversity over time.

Honest Assessment

What works: generates valid, playable MIDI that imports cleanly into DAWs like GarageBand, FL Studio, and BandLab. Genre conditioning is demonstrably effective at 89% precision. Handles full audio-to-MIDI input via Basic Pitch. Runs in under 10 seconds on GPU.

What doesn't: no guarantee of harmonic correctness — chord progressions can clash with the input melody. No song structure (no verse/chorus). Only three genres supported. Polyphonic or harmonised vocal input isn't handled reliably.